

Procesamiento Adaptativo de Señales

Juan E. Cousseau

Departamento de Ingeniería Eléctrica y Computadoras

Universidad Nacional del Sur

Contenidos

- **Conceptos básicos.**
- **Filtro de Wiener.**
- **Filtro de Kalman.**
- **Menor cuadrado mínimo (LMS)**
- **Cuadrados mínimos recursivo (RLS)**
- **Tracking e implementación.**
- **Aprendizaje automático.**

Menor Cuadrado Mínimo (LMS)

Método de Steepest Descent

- Algoritmo Steepest Descent
- Estabilidad del algoritmo

Algoritmo LMS

- Ajuste de los coeficientes
- Estabilidad y desempeño del algoritmo LMS
- El algoritmo LMS normalizado

Filtrado adaptativo en el dominio frecuencia

- Filtrado adaptativo en bloques
- Algoritmo adaptativo con preprocesamiento
- Clasificación de algoritmos

Algoritmo Least Mean Square (LMS)

- Se discutirá la teoría relacionada con el algoritmo denominado *least-mean-square* (LMS) por sus autores, Widrow y Hopf (1960).
- Es un miembro importante de la familia de *algoritmos de gradiente estocástico* (distingue al LMS del método SD que utiliza un gradiente determinístico).
- Una característica significativa del algoritmo LMS es su *simplicidad* (estándar frente a otros algoritmos de filtrado adaptativo).
- Se discutirán aspectos relacionados con el análisis de convergencia del algoritmo LMS, comenzando el estudio con una revisión de su estructura y operación.

Algoritmo Least Mean Square (LMS) ... Preliminares

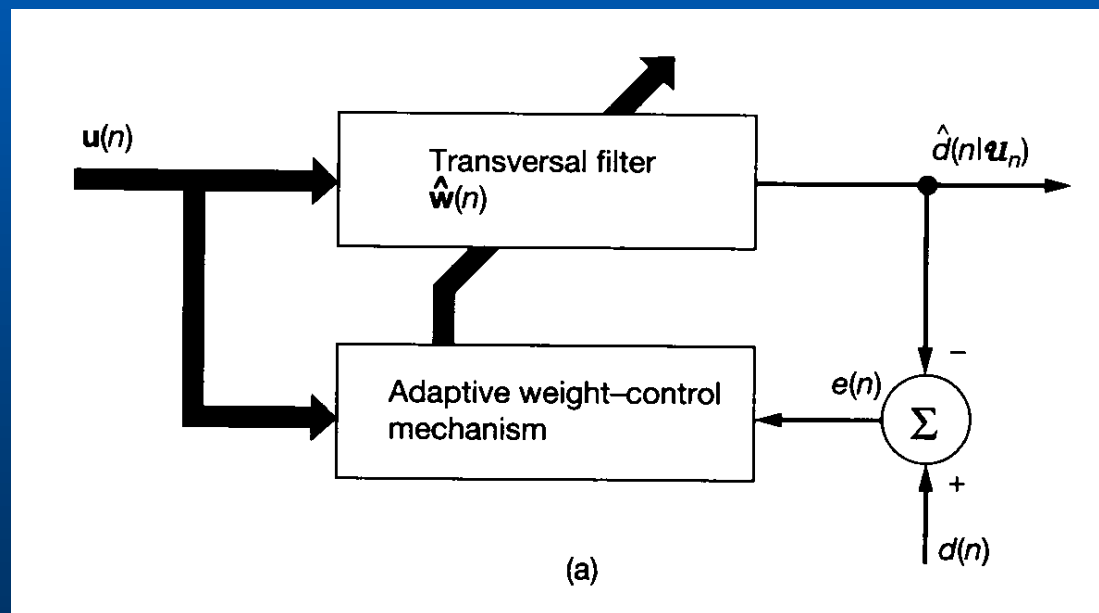
El LMS es un algoritmo de filtrado adaptativo lineal que está formado por dos procesos básicos:

- *Un proceso de filtrado*: que involucra (a) calcular la salida del filtro transversal producida por la entrada y (b) generación de un error de estimación comparando la salida con una respuesta deseada.
- *Un proceso adaptivo*: que involucra el ajuste automático de los coeficientes del filtro de acuerdo con el error de estimación.

La combinación de estos dos procesos trabajando juntos constituye un *lazo de realimentación* alrededor del algoritmo LMS.

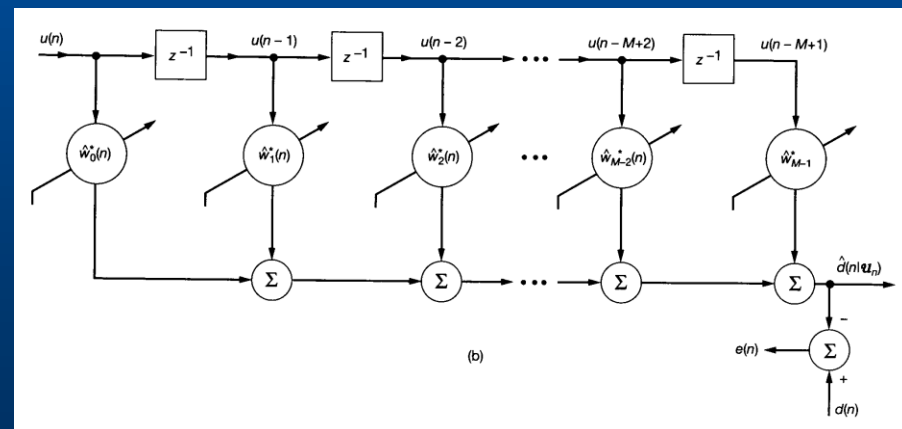
Algoritmo Least Mean Square (LMS) ... Preliminares

La combinación de estos dos procesos trabajando juntos constituye un *lazo de realimentación* alrededor del algoritmo LMS.



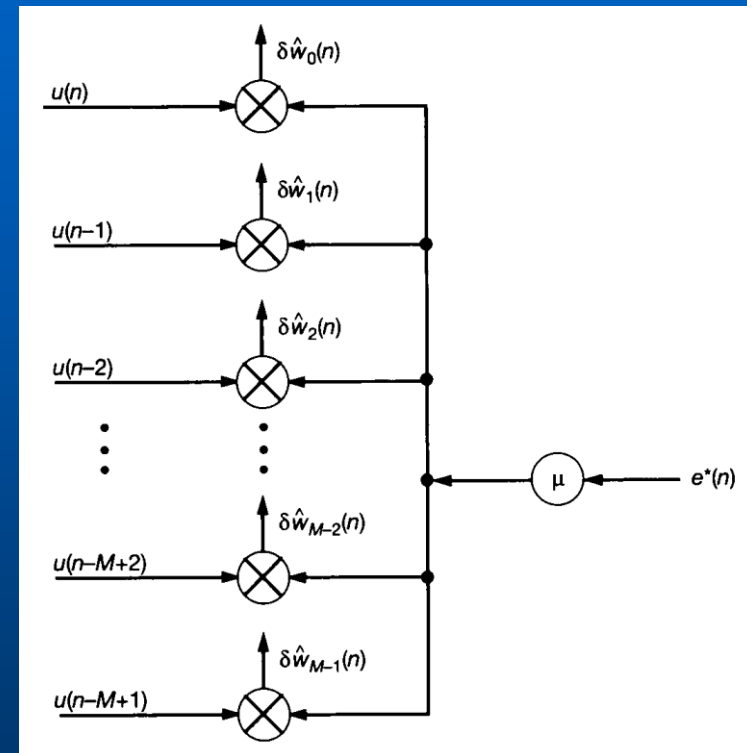
Algoritmo Least Mean Square (LMS) ... Preliminares

- Algunos detalles del filtro transversal: las muestras de la entrada $u(n)$, $u(n-1)$, ..., $u(n-M+1)$ forman $\mathbf{u}(n)$ ($M \times 1$), que genera espacio multidimensional $\mathcal{U}_n$. En forma similar, los coeficientes $\hat{w}_0(n)$, $\hat{w}_1(n)$, ..., $\hat{w}_{M-1}(n)$ forman el vector de coeficientes $\hat{\mathbf{w}}(n)$.
- El valor calculado para $\hat{\mathbf{w}}(n)$ usando el algoritmo LMS representa una estimación cuyo valor esperado se aproxima a la solución de Wiener $\mathbf{w}_o$ (para un contexto ESA) cuando el número de iteraciones $n \rightarrow \infty$
- El filtro transversal produce $\hat{d}(n|\mathcal{U}_n)$ como estimación de la respuesta deseada $d(n)$. Esto permite definir el error de estimación $e(n)$. $e(n)$ y $\mathbf{u}(n)$ se aplican al mecanismo de control y así se cierra el lazo de realimentación alrededor de los coeficientes.



Algoritmo Least Mean Square (LMS) ... Preliminares ...

- Detalles del mecanismo de control de los coeficientes: una versión escalar del producto interno de $e(n)$ y las entradas $u(n-k)$, para $k=0, 1, \dots, M-1$.
- El resultado obtenido define la corrección $\delta \hat{w}_k(n)$ aplicada al coeficiente $\hat{w}_k(n)$ en la iteración $n+1$.
- El factor de escala μ , y se denomina *factor de convergencia*.



Algoritmo Least Mean Square (LMS) ...

Preliminares ...

- Comparando el algoritmo LMS con el del método SD es posible concluir que el primero utiliza $u(n-k)e^*(k)$ como una estimación del elemento k del vector gradiente $\nabla J(n)$ del método SD. En consecuencia, el cálculo recursivo de cada coeficiente por medio del LMS sufre de un *ruido de gradiente*.
- En este estudio se supone que $u(n)$ y $d(n)$ se obtienen de un contexto ESA conjuntamente. Para ese contexto, a diferencia del SD (trayectoria determinística de $w(n)$ a la solución de Wiener w_o), en el LMS, por la presencia del ruido de gradiente, $\hat{w}(n)$ converge a un entorno (aleatorio) alrededor del punto de mínimo de la superficie de desempeño.

Algoritmo Least Mean Square (LMS) ...

Preliminares ...

- Anteriormente señalamos que el algoritmo LMS involucra una operación de realimentación, la cual introduce el aspecto relacionado de *estabilidad*. En este contexto, un criterio significativo es requerir que

$$J(n) \rightarrow J(\infty), \quad \text{cuando } n \rightarrow \infty$$

donde $J(n)$ es el MSE producido por el algoritmo LMS en el instante n , y su valor final $J(\infty)$ es una constante. Un algoritmo que satisface este requerimiento se dice *convergente en sentido medio cuadrático*.

- Para que el algoritmo LMS satisfaga este criterio, el factor de convergencia μ debe satisfacer ciertas condiciones relacionadas con la estructura de autovalores de la matriz correlación de la entrada.

Algoritmo Least Mean Square (LMS) ...

Preliminares ...

- $J_{ex}(\infty) = J(\infty) - J_{min}$ se denomina *exceso de error medio cuadrático*. Esta diferencia es el precio por usar un mecanismo adaptativo para controlar los coeficientes LMS en lugar de un mecanismo determinístico.
- $\mathcal{M} = J_{ex}(\infty)/J_{min}$, se denomina *desajuste*, y representa una medida de cuán lejos la solución de estado estacionario obtenida con el LMS está de la solución de Wiener.
- *Es importante tener en cuenta, que $\mathcal{M}$ puede controlarse.* En particular, el lazo de realimentación que actúa sobre los coeficientes se comporta como un *filtro pasabajos*, cuya constante de tiempo promedio es inversamente proporcional a μ . Con μ pequeño, el proceso adaptativo se hace lento y los efectos del ruido de gradiente sobre los coeficientes se suavizan. Esto a su vez conduce a una reducción del desajuste.

Algoritmo Least Mean Square (LMS) ...

Ajuste de los coeficientes

- Las mediciones exactas del vector gradiente no están en general disponibles y debe estimarse.
- Para una estimación de $\nabla J(n)$, la estrategia obvia es sustituir estimaciones de $\mathbf{R}$ y $\mathbf{p}$ en $\nabla J(n) = -2\mathbf{p} + 2\mathbf{R}\mathbf{w}(n)$.
- La **elección más simple** de los estimadores para $\mathbf{R}$ y $\mathbf{p}$ es utilizar **valores instantáneos**, dados por

$$\hat{\mathbf{R}}(n) = \mathbf{u}(n)\mathbf{u}^H(n) \quad \hat{\mathbf{p}}(n) = \mathbf{u}(n)d^*(n)$$

- En consecuencia, la estimación instantánea del vector gradiente será

$$\hat{\nabla} J(n) = -2\mathbf{u}(n)d^*(n) + 2\mathbf{u}(n)\mathbf{u}^H(n)\hat{\mathbf{w}}(n)$$

Algoritmo Least Mean Square (LMS) ... Ajuste de los coeficientes ...

- Sustituyendo esta ecuación en el algoritmo SD se obtiene

$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu \mathbf{u}(n)[d^*(n) - \mathbf{u}^H(n)\hat{\mathbf{w}}(n)]$$

lo que puede escribirse en base a tres relaciones básicas:

1. Salida del filtro: $y(n) = \hat{\mathbf{w}}^H(n)\mathbf{u}(n)$.
 2. Error de estimación: $e(n) = d(n) - y(n)$.
 3. Adaptación de los coeficientes: $\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu \mathbf{u}(n)e^*(n)$.
- El algoritmo LMS requiere $2M+1$ multiplicaciones complejas y $2M$ sumas complejas por iteración, i.e., la complejidad computacional es $O(M)$.

Algoritmo Least Mean Square (LMS) ...

Ejemplo 1: Modelo canónico del LMS complejo

- El algoritmo LMS descrito anteriormente es *complejo* en el sentido que la entrada y la salida así como los coeficientes son complejos. Definimos

Vector de entrada: $\mathbf{u}(n) = \mathbf{u}_I(n) + j\mathbf{u}_Q(n)$

Respuesta deseada: $d(n) = d_I(n) + jd_Q(n)$

Vector de coeficientes: $\hat{\mathbf{w}}(n) = \hat{\mathbf{w}}_I(n) + j\hat{\mathbf{w}}_Q(n)$

Salida del filtro: $y(n) = y_I(n) + jy_Q(n)$

Error de estimación: $e(n) = e_I(n) + je_Q(n)$

Algoritmo Least Mean Square (LMS) ...

Ejemplo 1: Modelo canónico del LMS complejo

- Usando las definiciones del algoritmo se tiene que

$$\begin{aligned}y_I(n) &= \hat{\mathbf{w}}_I^T(n) \mathbf{u}_I(n) - \hat{\mathbf{w}}_Q^T(n) \mathbf{u}_Q(n) \\y_Q(n) &= \hat{\mathbf{w}}_I^T(n) \mathbf{u}_Q(n) - \hat{\mathbf{w}}_Q^T(n) \mathbf{u}_I(n) \\e_I(n) &= d_I(n) - y_I(n) \\e_Q(n) &= d_Q(n) - y_Q(n) \\\hat{\mathbf{w}}_I(n+1) &= \hat{\mathbf{w}}_I(n) + \mu[e_I(n) \mathbf{u}_I(n) - e_Q(n) \mathbf{u}_Q(n)] \\\hat{\mathbf{w}}_Q(n+1) &= \hat{\mathbf{w}}_Q(n) + \mu[e_I(n) \mathbf{u}_Q(n) + e_Q(n) \mathbf{u}_I(n)]\end{aligned}$$

- Este modelo del LMS complejo es equivalente a un *conjunto de cuatro LMS reales con acoplamiento cruzado entre ellos*.

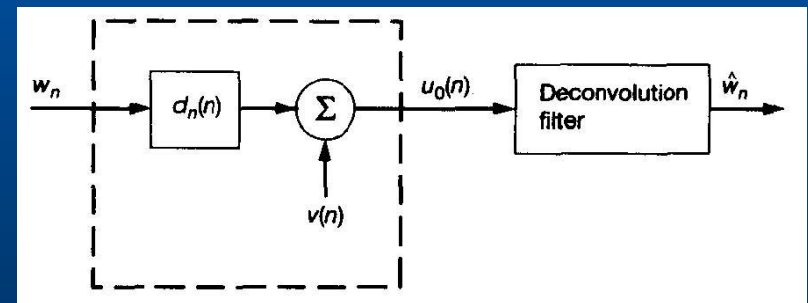
Algoritmo Least Mean Square (LMS) ...

Ejemplo 2: Deconvolución predictiva

- La idea de *deconvolución predictiva* es útil para el procesamiento de sismogramas de reflexión. Se supondrá que los datos sísmicos son ESA durante la ventana utilizada para generar la operación de deconvolución.
- Dos suposiciones: la salida del sistema de reflexiones sísmicas $d_0(n)$ sigue un modelo AR (solo polos); la excitación correspondiente w_n tendrá características de un proceso de ruido blanco.
- Entonces

$$u_0(n) = w_n * d_0(n)$$

$$w_n = u_0(n) * \text{inv}[d_0(n)]$$



$$d_0(n) = - \sum_{k=1}^M a_k d_0(n-k), \quad D_0(z) = \frac{1}{A(z)}, \quad A(z) = (1 + a_1 z^{-1} + \dots + a_M z^{-M})$$

Algoritmo Least Mean Square (LMS) ...

Ejemplo 2: Deconvolución predictiva ...

- Un operador M -dimensional $\hat{\mathbf{w}}(n)$ se utiliza para generar una predicción de la traza a partir de los datos:

$$\hat{u}_0(n + \Delta) = \hat{\mathbf{w}}^T(n) \mathbf{u}_0(n)$$

donde $\hat{\mathbf{w}}(n) = [\hat{w}_0(n), \hat{w}_1(n), \dots, \hat{w}_{M-1}(n)]^T$

$$\mathbf{u}_0(n) = [u_0(n), u_0(n-1), \dots, u_0(n-M+1)]^T$$

y $\Delta \geq 1$ es la *profundidad de predicción*.

La traza deconvolucionada $y(n)$ se define como la diferencia entre la entrada y la traza predicha,

$$y(n) = u_0(n) - \hat{u}_0(n)$$

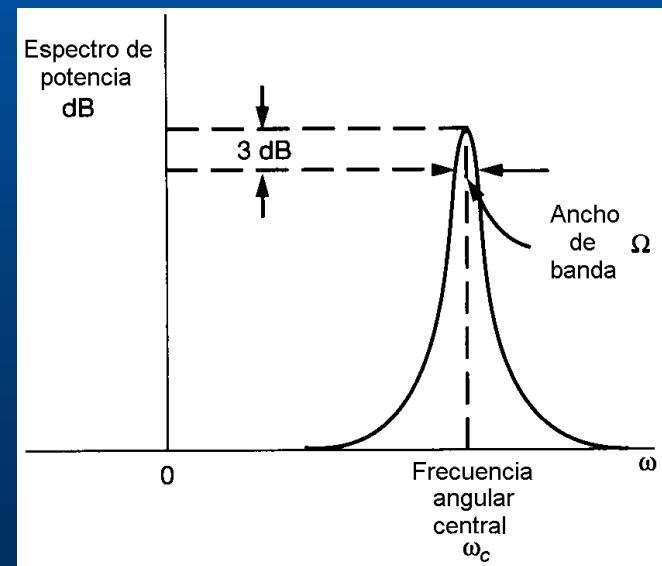
Finalmente, la ecuación de predicción del algoritmo LMS será

$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu y(n+\Delta) \mathbf{u}_0(n)$$

Algoritmo Least Mean Square (LMS) ...

Ejemplo 3: Medición de frecuencia instantánea

- Uso del algoritmo LMS como base para estimar el contenido de frecuencia de una señal de *banda estrecha*.
- Relación entre tres ideas básicas: un modelo AR para describir un proceso estocástico, un predictor para analizar el proceso y el algoritmo LMS para estimar los parámetros AR.
- Definimos una señal de banda estrecha como aquella cuyo ancho de banda Ω es pequeño comparado con la frecuencia central ω_c .
- Una señal modulada en frecuencia (FM) es un ejemplo de señal de banda estrecha. La *frecuencia instantánea* (derivada de la fase con respecto al tiempo) de una FM varía linealmente con la señal moduladora.



Algoritmo Least Mean Square (LMS) ...

Ejemplo 3: Medición de frecuencia instantánea ...

- Considerando un proceso de banda estrecha $u(n)$ generado por un modelo AR variante en el tiempo de orden M , dado por

$$u(n) = - \sum_{k=1}^M a_k(n) u(n-k) + \nu(n)$$

donde $a_k(n)$ son los parámetros variantes en el tiempo del modelo y $\nu(n)$ es un proceso de ruido blanco con media cero y varianza $\sigma_\nu^2(n)$.

- El espectro de potencia AR (variante en el tiempo) del proceso $u(n)$ está dado por

$$S_{AR}(\omega; n) = \frac{\sigma_\nu^2(n)}{\left| 1 + \sum_{k=1}^M a_k(n) e^{-j\omega k} \right|^2}, \quad -\pi < \omega \leq \pi$$

- Notar que un proceso AR cuyos polos está próximos al círculo unitario del plano Z tiene las características de un proceso de banda estrecha.

Algoritmo Least Mean Square (LMS) ...

Ejemplo 3: Medición de frecuencia instantánea ...

- Para estimar los parámetros del modelo se utiliza un filtro transversal como un predictor lineal de orden M , con coeficientes $\hat{w}_k(n)$, $k=1, 2, \dots, M$. Los coeficientes se adaptan continuamente con la señal de entrada $u(n)$. En particular, utilizamos el algoritmo LMS dado por

$$\hat{w}_k(n+1) = \hat{w}_k(n) + \mu u(n-k) f_M(n), \quad k = 1, 2, \dots, M$$

donde $f_M(n)$ es el *error de predicción* dado por

$$f_M(n) = u(n) - \sum_{k=1}^M \hat{w}_k(n) u(n-k)$$

- Los coeficientes del predictor adaptativo están relacionados con los parámetros del modelo AR por

$$-\hat{w}_k(n) = \text{estimación de } a_k(n) \text{ en el instante } n, \text{ para } k = 1, 2, \dots, M$$

Algoritmo Least Mean Square (LMS) ...

Ejemplo 3: Medición de frecuencia instantánea ...

- Además, la potencia promedio del error de predicción $f_M(n)$ provee una estimación de la varianza del ruido $\sigma_v^2(n)$.
- El objetivo es localizar la frecuencia de una señal de banda estrecha. En consecuencia, en lo siguiente se ignorará la estimación de $\sigma_v^2(n)$. Específicamente, utilizaremos los coeficientes del predictor adaptativo para definir una *función frecuencia variante en el tiempo* definida por

$$F(\omega; n) = \frac{1}{\left| 1 + \sum_{k=1}^M a_k(n) e^{-j\omega k} \right|^2}$$

- La diferencia esencial entre $F(\omega; n)$ y $S_{AR}(\omega; n)$ reside en sus factores de escala en el numerador.

Algoritmo Least Mean Square (LMS) ...

Ejemplo 3: Medición de frecuencia instantánea ...

Es posible utilizar $F(\omega; n)$ para medir la frecuencia instantánea de una señal de FM $u(n)$, asumiendo que las siguientes suposiciones sean válidas:

- El predictor adaptativo ha estado en operación por tiempo suficiente para asegurar que cualquier transitorio causado por la inicialización de los coeficientes ha finalizado.
- El factor de convergencia μ se eligió correctamente para que el predictor adaptativo funcione adecuadamente, o sea, el error de predicción $f_M(n)$ es pequeño para todo n .
- La señal moduladora es esencialmente constante sobre el rango de muestras del predictor adaptativo, el cual se extiende desde $(n-M)$ hasta $(n-1)$.

Algoritmo Least Mean Square (LMS) ... Estabilidad y desempeño

Cuando el factor de convergencia μ es pequeño, el algoritmo LMS se comporta como un **filtro pasabajos con una frecuencia de corte pequeña**.

Basándose en este comportamiento, la teoría de aprendizaje estadístico del algoritmo aborda los siguientes problemas:

- Condición impuesta a μ para la convergencia (tiempo de transitorio).
- Desajuste debido al ruido de gradiente (varianza final).
- Eficiencia del algoritmo (compromisos).

Los tres problemas son de importancia práctica.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño ...

- En el estudio de las propiedades, conviene trabajar con el vector de error en los coeficientes antes que con el vector de coeficientes

$$\epsilon(n) = \hat{\mathbf{w}}(n) - \mathbf{w}_o$$

- En base al algoritmo LMS es posible obtener

$$\epsilon(n+1) = [\mathbf{I} - \mu \mathbf{u}(n) \mathbf{u}^H(n)] \epsilon(n) - \mu \mathbf{u}(n) e_o^*(n)$$

donde $e_o(n)$ es el *error de estimación producido con la solución óptima de Wiener*, dado por

$$e_o(n) = d(n) - \mathbf{w}_o^H \mathbf{u}(n)$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Método de promediación directa

- La ecuación anterior es una ecuación a diferencias estocástica en el vector de error de los coeficientes $\epsilon(n)$ con la siguiente característica

Una matriz de sistema $[I - \mu \mathbf{u}(n)\mathbf{u}^H(n)]$, aproximadamente igual a I para todo n , previendo que μ sea suficientemente pequeño.

- De acuerdo con este método (Kushner), la solución a esa ecuación a diferencias estocástica, operando bajo la suposición de un μ pequeño, es próxima a la solución de otra ecuación a diferencias estocástica cuya matriz de sistema es

$$E[I - \mu \mathbf{u}(n)\mathbf{u}^H(n)] = I - \mu \mathbf{R}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Método de promediación directa ...

- Específicamente, es posible reemplazar aquella ecuación con

$$\epsilon_0(n+1) = (\mathbf{I} - \mu \mathbf{R})\epsilon_0(n) - \mu \mathbf{u}(n)e_o^*(n)$$

- Este método es una aproximación heurística razonable, dado que está basado en la idea que, para μ pequeño, la aleatoriedad de $\epsilon_0(n)$ tiende a desaparecer.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Método de análisis iterativo

A través de un procedimiento iterativo (Butterweck), la solución de la ecuación de promediación directa anterior se usa como punto de partida para generar el conjunto completo de soluciones de la ecuación a diferencias estocástica original.

La precisión de la solución así obtenida mejora con el incremento de las iteraciones.

Se parte de

$$\epsilon(n) = \epsilon_0(n) + \epsilon_1(n) + \epsilon_2(n) + \dots$$

donde $\epsilon_0(n)$ corresponde a la solución de promediación directa, para $\mu \rightarrow 0$, y también $\epsilon_1(n) + \epsilon_2(n) + \dots$ son correcciones de orden superior para $\mu > 0$.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Método de análisis iterativo

Definiendo la matriz de media 0 dada por $\mathbf{P}(n) = \mathbf{u}(n)\mathbf{u}^H(n) - \mathbf{R}$, entonces en la ecuación a diferencias original

$$\begin{aligned}\epsilon_0(n+1) + \epsilon_1(n+1) + \epsilon_2(n+1) + \cdots &= (\mathbf{I} - \mu\mathbf{R})[\epsilon_0(n) + \epsilon_1(n) + \epsilon_2(n) + \cdots] \\ &+ \mu\mathbf{P}[\epsilon_0(n) + \epsilon_1(n) + \epsilon_2(n) + \cdots] \\ &- \mu\mathbf{u}(n)e_o^*(n)\end{aligned}$$

o, en forma de ecuaciones acopladas

$$\epsilon_i(n+1) = (\mathbf{I} - \mu\mathbf{R})\epsilon_i(n) + \mathbf{f}_i(n)$$

donde

$$\mathbf{f}_i(n) = \begin{cases} -\mu\mathbf{u}e_o^*(n), & i = 0 \\ -\mu\mathbf{P}(n)\epsilon_{i-1}(n) & i = 1, 2, \dots \end{cases}$$

Tal que la solución de la ecuación i -ésima del proceso iterativo depende de la de la solución $i-1$.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Método de análisis iterativo

En particular, el análisis del algoritmo LMS se reduce a un estudio de la transmisión de un proceso estocástico estacionario a través de un filtro pasabajos con una frecuencia de corte extremadamente baja a medida que el parámetro de tamaño de paso $\mu \rightarrow 0$.

Con base en la ecuación a diferencias anterior, la matriz correlación del vector de error en los coeficientes será

$$\mathbf{K}(n) = E\{\boldsymbol{\epsilon}(n)\boldsymbol{\epsilon}^H(n)\} = \sum_i \sum_k E\{\epsilon_i(n)\epsilon_k^H(n)\}, \quad (i, k) = 0, 1, \dots$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Método de análisis iterativo

Expandiendo en base a las iteraciones y agrupando términos de igual μ ,

$$\mathbf{K}(n) = \mathbf{K}_0(n) + \mu \mathbf{K}_1(n) + \mu^2 \mathbf{K}_2(n) + \cdots$$

donde

$$\mu^j \mathbf{K}_j(n) = \begin{cases} E\{\epsilon_0(n) \epsilon_0^H(n)\} & j = 0 \\ \sum_i \sum_k E\{\epsilon_i(n) \epsilon_k^H(n)\}, & (i, k) \geq 0, i + k = 2j - 1 \text{ ó } 2j \end{cases}$$

Teniendo en cuenta que los sucesivos errores $\epsilon_i(n)$ deben ser de magnitud cada vez menor (son correcciones sucesivas), esta descomposición indica que existe una gran decorrelación entre las $\mathbf{K}_j(n)$.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Suposiciones simplificadoras

Suposición 1: El parámetro de tamaño de paso μ es pequeño, por lo que el algoritmo LMS actúa como un filtro pasabajos con una frecuencia de corte baja. De esta forma, es posible utilizar los términos $\epsilon_0(n)$ y $\mathbf{K}_0(n)$ como aproximaciones de $\epsilon(n)$ y $\mathbf{K}(n)$.

Suposición 2: El mecanismo físico para generar la respuesta deseada $d(n)$ se describe mediante un modelo de regresión lineal múltiple que coincide exactamente con el filtro de Wiener:

$$d(n) = \mathbf{w}_o^H \mathbf{u}(n) + e_o(n)$$

Suposición 3: El vector de entrada $\mathbf{u}(n)$ y la respuesta deseada $d(n)$ son conjuntamente gaussianos.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Modos naturales

Con la suposición 1, se parte de

$$\epsilon_0(n+1) = (\mathbf{I} - \mu \mathbf{R})\epsilon_0(n) + \mathbf{f}_0(n), \quad \mathbf{f}_0(n) = -\mu \mathbf{u}(n)e_o^*(n)$$

Para simplificar es posible usar $\mathbf{Q}^H \mathbf{R} \mathbf{Q} = \mathbf{\Lambda}$, $\mathbf{Q} \mathbf{Q}^H = \mathbf{I}$ y se obtiene

$$\mathbf{v}(n+1) = (\mathbf{I} - \mu \mathbf{\Lambda})\mathbf{v}(n) + \phi(n) \quad \mathbf{v}(n) = \mathbf{Q}^H \epsilon_0(n), \quad \phi(n) = \mathbf{Q}^H \mathbf{f}_0(n)$$

Y usando las suposiciones 2 o 3,

$$E\{\phi(n)\} = 0, \forall n, \quad E\{\phi(n)\phi^H(n)\} = \mu^2 J_{min} \mathbf{\Lambda}$$

Entonces, en forma escalar, la ecuación de los modos naturales del LMS pone en evidencia la diferencia con el método SD, el término forzado $\phi_k(n)$

$$v_k(n+1) = (1 - \mu \lambda_k)v_k(n) + \phi_k(n), \quad k = 1, 2, \dots, M$$

Algoritmo Least Mean Square (LMS) ... Estabilidad y desempeño: Modos naturales...

Para una condición inicial $v_k(0)$, la solución de esta ecuación es

$$v_k(n) = (1 - \mu\lambda_k)^n v_k(0) + \sum_{i=0}^{n-1} (1 - \mu\lambda_k)^{n-1-i} \phi_k(i)$$

tal que

$$\begin{aligned} E\{v_k(n)\} &= v_k(1 - \mu\lambda_k)^n, \\ E\{|v_k(n)|^2\} &= \frac{\mu J_{min}}{2 - \mu\lambda_k} + (1 - \mu\lambda_k)^{2n} \left(|v_k(0)|^2 - \frac{\mu J_{min}}{2 - \mu\lambda_k} \right) \end{aligned}$$

Siempre que el parámetro de tamaño de paso de un algoritmo LMS sea pequeño, los modos naturales del algoritmo **ejecutan un movimiento Browniano** alrededor de algunos valores fijos, con media y varianza de los modos naturales así definidas.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Curvas de aprendizaje

Es común utilizar curvas de aprendizaje promedio para estudiar el comportamiento estadístico de los algoritmos de filtrado adaptativo. En particular:

1. La *curva de aprendizaje de error cuadrático medio* (MSE), promedios del error cuadrático de estimación,

$$J(n) = E\{|e(n)|^2\}$$

2. La *curva de aprendizaje de desviación cuadrática media* (MSD), que se basa en el promedio conjunto de la desviación cuadrática del error en los coeficientes, n .

$$\mathcal{D}(n) = E\{|\epsilon(n)|^2\}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Curvas de aprendizaje ...

- A diferencia del SD, tanto $J(n)$ como $\mathcal{D}(n)$ en el LMS dependen de la iteración porque $e(n)$ y $\epsilon(n)$ son *procesos no estacionarios*.
- El error de estimación, con la suposición 1, es

$$\begin{aligned}e(n) &= d(n) - \hat{\mathbf{w}}^H(n)\mathbf{u}(n) = d(n) - \mathbf{w}_o^H \mathbf{u}(n) - \boldsymbol{\epsilon}^H(n)\mathbf{u}(n) \\ &= e_o(n) - \boldsymbol{\epsilon}^H(n)\mathbf{u}(n) \cong e_o(n) - \boldsymbol{\epsilon}_0^H(n)\mathbf{u}(n)\end{aligned}$$

- Luego, usando esta para obtener $J(n)$ y con la suposición 2 se obtiene

$$\begin{aligned}J(n) &= E[|e(n)|^2] \\ &= E[(e_o(n) - \boldsymbol{\epsilon}_0^H(n)\mathbf{u}(n))(e_o^*(n) - \mathbf{u}^H(n)\boldsymbol{\epsilon}_0(n))] \\ &= J_{min} + E[\boldsymbol{\epsilon}_0^H(n)\mathbf{u}(n)\mathbf{u}^H(n)\boldsymbol{\epsilon}_0(n)]\end{aligned}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Curvas de aprendizaje...

- La esperanza de esta última ecuación está asociada con una variable aleatoria escalar representada por un producto de vectores triple, y como la traza de un escalar es el escalar mismo se tiene que

$$\begin{aligned} E[\epsilon_0^H(n) \mathbf{u}(n) \mathbf{u}^H(n) \epsilon_0(n)] &= E[\text{tr}\{\epsilon_0^H(n) \mathbf{u}(n) \mathbf{u}^H(n) \epsilon_0(n)\}] \\ &= E[\text{tr}\{\mathbf{u}(n) \mathbf{u}^H(n) \epsilon_0(n) \epsilon_0^H(n)\}] \\ &= \text{tr}\{E[\mathbf{u}(n) \mathbf{u}^H(n) \epsilon_0(n) \epsilon_0^H(n)]\} \end{aligned}$$

- Invocando el *método de promediación directa* nuevamente es posible reducir esta expresión a

$$\begin{aligned} E[\epsilon_0^H(n) \mathbf{u}(n) \mathbf{u}^H(n) \epsilon_0(n)] &= \text{tr}\{E[\mathbf{u}(n) \mathbf{u}^H(n)] E[\epsilon_0(n) \epsilon_0^H(n)]\} \\ &= \text{tr}\{\mathbf{R} \mathbf{K}_0(n)\} \end{aligned}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Curvas de aprendizaje ...

- De esta forma:

$$J(n) \cong J_{min} + tr\{\mathbf{R}\mathbf{K}_0(n)\}$$

- Esta ecuación indica que para todo n , el MSE en el algoritmo LMS consiste en dos componentes: el MMSE J_{min} y una componente dependiente del comportamiento transitorio de $\mathbf{K}_0(n)$.
- Dado que esta última componente es positiva definida para todo n , *el algoritmo LMS siempre produce un MSE $J(n)$ en exceso del MMSE.*
- Definimos formalmente el *exceso de error medio cuadrático* como

$$J_{ex}(n) = J(n) - J_{min} \cong tr[\mathbf{R}\mathbf{K}_0(n)]$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Curvas de aprendizaje ...

- También, usando la definición de $\mathbf{K}_0(n)$ se obtiene

$$\begin{aligned} J_{ex}(n) &\cong \text{tr}[\mathbf{R}E\{\boldsymbol{\epsilon}_0(n)\boldsymbol{\epsilon}_0^H(n)\}] \\ &= \text{tr}[\mathbf{R}E\{\mathbf{Q}\mathbf{v}(n)\mathbf{v}^H(n)\mathbf{Q}\}] \\ &= E\{\text{tr}[\mathbf{R}\mathbf{Q}\mathbf{v}(n)\mathbf{v}^H(n)\mathbf{Q}]\} \\ &= E\{\text{tr}[\mathbf{v}^H(n)\mathbf{Q}^H\mathbf{R}\mathbf{Q}\mathbf{v}(n)]\} \\ &= E\{\text{tr}[\mathbf{v}^H(n)\boldsymbol{\Lambda}\mathbf{v}(n)]\} \\ &= \sum_{k=1}^M \lambda_k E\{|v_k|^2\} \end{aligned}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Curvas de aprendizaje ...

Usando también la suposición 1, el MSD será

$$\mathcal{D}(n) \cong E\{\|\epsilon_0(n)\|^2\} = E\{\|\mathbf{v}(n)\|^2\} = \sum_{k=1}^M E\{|v_k(n)|^2\}$$

Usando ahora las expresiones de $J_{ex}(n)$ y $\mathcal{D}(n)$, y los autovalores máximo y mínimo de $\mathbf{R}$, se tiene que

$$\frac{J_{ex}(n)}{\lambda_{min}} \geq \mathcal{D}(n) \geq \frac{J_{ex}(n)}{\lambda_{max}}$$

Entonces como $\mathcal{D}(n)$ está acotada por las iteraciones de $J_{ex}(n)$ solo interesa poner atención en este último.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Comportamiento transitorio

Teniendo en cuenta el análisis de los modos naturales, el factor $(1 - \mu\lambda_k)^n$ gobierna la convergencia del modo natural k del LMS en la iteración n .

La condición necesaria para que ese factor tienda a cero es $-1 < 1 - \mu\lambda_k < 1$, o sea

$$0 < \mu < \frac{2}{\lambda_{max}}$$

Eso implicaría que

$$E\{v_k(n)\} \rightarrow 0, \text{ para } n \rightarrow \infty, \quad \text{ó} \quad E\{\hat{w}(n)\} \rightarrow \hat{w}_o, \text{ para } n \rightarrow \infty$$

Sin embargo, esto no es práctico ya que una secuencia variables aleatorias de media asintótica 0, no necesariamente tiende a cero.

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Comportamiento transitorio ...

Un criterio más adecuado es la *convergencia en media cuadrática*, asociada al promedio usual de las curvas de aprendizaje.

Usando los resultados de los modos naturales en $J_{ex}(n)$ se obtiene

$$J(n) = J_{min} + \mu J_{min} \sum_{k=1}^M \frac{\lambda_k}{2 - \mu \lambda_k} + \sum_{k=1}^M \lambda_k \left(|v_k(0)|^2 - \frac{\mu J_{min}}{2 - \mu \lambda_k} \right) (1 - \mu \lambda_k)^{2n}$$

Entonces, para μ suficientemente pequeño, la curva de aprendizaje del MSE disminuye sin oscilaciones al siguiente valor

$$J(\infty) = J_{min} + \mu J_{min} \sum_{k=1}^M \frac{\lambda_k}{2 - \mu \lambda_k} \cong J_{min} + \frac{\mu J_{min}}{2} \sum_{k=1}^M \lambda_k$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Desajuste

Retornando a la definición de desajuste, $\mathcal{M} = J_{ex}(\infty)/J_{min}$ y con el resultado anterior

$$\mathcal{M} = \frac{\mu}{2} \sum_{k=1}^M \lambda_k = \frac{\mu}{2} \text{tr}[\mathbf{R}]$$

Para un proceso de entrada estacionario

$$\text{tr}[\mathbf{R}] = Mr(0) = E\{\|\mathbf{u}(n)\|^2\}$$

De manera que el desajuste se puede obtener en forma práctica como

$$\mathcal{M} = \frac{\mu}{2} \times \text{potencia total de entrada}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Eficiencia

La velocidad de convergencia a la solución de Wiener es una medida de la **eficiencia estadística** del LMS. Esa velocidad está determinada por las constantes de tiempo en la respuesta transitoria.

Partiendo de la ecuación de modos naturales, y con μ suficientemente pequeño, la *razón de la serie geométrica* que controla $J(n)$ es

$$t_k(n) = (1 - \mu\lambda_k)^{2n} = (1 - 2\mu\lambda_k + \mu^2\lambda_k^2)^n \cong (1 - 2\mu\lambda_k)^n$$

Entre 2 iteraciones sucesivas se puede definir la **constante de tiempo del autovalor k** como

$$\frac{1}{\tau_{mse,k}} = t(0) - t(1) \cong 2\mu\lambda_k, \quad \text{ó} \quad \tau_{mse,k} = \frac{1}{2\mu\lambda_k}$$

Algoritmo Least Mean Square (LMS) ...

Estabilidad y desempeño: Eficiencia ...

1. A medida que los coeficientes del LMS convergen hacia la solución de Wiener, la curva de aprendizaje experimenta una progresión geométrica alrededor del MMSE, J_{min} .
2. El LMS es sensible a la dispersión de autovalores de R .
3. En las peores condiciones, la velocidad de convergencia estará dominada por el autovalor más pequeño, λ_{min} ; la constante de tiempo más lenta para la curva de aprendizaje será

$$\tau_{mse,min} = \frac{1}{2\mu\lambda_{min}}$$

Algoritmo Least Mean Square (LMS) ...

Algunas reglas prácticas ...

- Usando un valor promedio de λ_{av} en $\tau_{mse,av}$ se obtiene

$$\mathcal{M} \cong \frac{\mu M \lambda_{av}}{2} = \frac{M}{4\tau_{mse,av}}$$

de lo que es posible hacer las siguientes observaciones:

1. El desajuste $\mathcal{M}$ aumenta linealmente con la longitud del filtro M para un valor fijo de $\tau_{mse,av}$
2. El *tiempo de convergencia* del algoritmo LMS (o sea el que toma para eliminar todo el transitorio) es proporcional a la constante de tiempo promedio $\tau_{mse,av}$. Además $\mathcal{M}$ es inversamente proporcional a este tiempo de convergencia.
3. El desajuste $\mathcal{M}$ es directamente proporcional a μ , mientras que la constante de tiempo promedio es inversamente proporcional a μ . De aquí que es necesario satisfacer el compromiso entre bajo desajuste y tiempo de convergencia corto a través del factor de convergencia μ .

Algoritmo Least Mean Square (LMS) ...

Parámetros: M = número de coeficientes

μ = factor de convergencia tal que

$$0 < \mu < \frac{2}{\text{potencia promedio de } u(n)}$$

Datos: $\mathbf{u}(n)$ vector de entradas ($M \times 1$).

$d(n)$ respuesta deseada en el instante n .

Inicialización

$$\hat{\mathbf{w}}(0) = \mathbf{0}$$

Para cada n , $n = 1, 2, \dots$, calcular

$$e(n) = d(n) - \hat{\mathbf{w}}^H(n) \mathbf{u}(n)$$

$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu \mathbf{u}(n) e^*(n)$$

Algoritmo Least Mean Square (LMS) ...

Robustez del algoritmo LMS

- El desarrollo del algoritmo LMS presentado se realizó de forma heurística, partiendo del SD como base para el cálculo de la solución de Wiener de un filtro transversal adaptativo.
- Sin embargo, al utilizar estimaciones instantáneas de R y p se pierde la relación con la estimación de MSE de Wiener.
- Por lo tanto, si una única ejecución del algoritmo LMS no es óptima en el sentido de mínimos cuadrados, ¿cuál es entonces el criterio real que determina su óptima performance?
- La respuesta a esta pregunta fundamental reside en el llamado criterio H^∞ (o minimax), del campo del control robusto.

Algoritmo Least Mean Square (LMS) ...

Robustez del algoritmo LMS

Partiendo del modelo de regresión múltiple

$$d(i) = \mathbf{w}_o^H \mathbf{u}(i) + \nu(i), \quad i = 0, 1, \dots, n$$

Se busca una estimación recursiva para estimar $\mathbf{w}_o$. El análisis buscara encontrar en base a que criterio la estimación del LMS $\hat{\mathbf{w}}(i)$ es óptima. Se incluyen las siguientes hipótesis:

1. Se dispone de μ tal que

$$0 < \mu \leq \frac{1}{\|\mathbf{u}(i)\|^2}, \quad 0 \leq i \leq n$$

2. Se propone una estimación $\hat{\mathbf{w}}(i)$ inicial. Esta estimación satisface

$$\frac{|\hat{\mathbf{w}}^H(i) \mathbf{u}(i) - \mathbf{w}_o^H \mathbf{u}(i)|^2}{\mu^{-1} \|\hat{\mathbf{w}}(i) - \mathbf{w}_o^H\|^2} \leq 1, \quad 0 \leq i \leq n$$

Algoritmo Least Mean Square (LMS) ...

Robustez del algoritmo LMS ...

Como aumentar el denominador de la expresión anterior no altera la desigualdad, es posible mostrar que, usando el algoritmo LMS, es posible modificar de la anterior como

$$\frac{|\hat{\mathbf{w}}^H(i)\mathbf{u}(i) - \mathbf{w}_o^H \mathbf{u}(i)|^2}{\mu^{-1}\|\hat{\mathbf{w}}(i) - \mathbf{w}_o^H\|^2 - \mu^{-1}\|\hat{\mathbf{w}}(i+1) - \mathbf{w}_o^H\|^2 + |\nu(i)|^2} \leq 1, \quad 0 \leq i \leq n$$

Suponiendo ahora

$$0 < \mu \leq \min_{0 \leq i \leq n} \frac{1}{\|\mathbf{u}(i)\|^2}$$

Se obtiene

$$\frac{\sum_{i=0}^n |\hat{\mathbf{w}}^H(i)\mathbf{u}(i) - \mathbf{w}_o^H \mathbf{u}(i)|^2}{\mu^{-1}\|\hat{\mathbf{w}}(i) - \mathbf{w}_o^H\|^2 + \sum_{i=0}^n |\nu(i)|^2} \leq 1,$$

Algoritmo Least Mean Square (LMS) ...

Robustez del algoritmo LMS

El numerador y denominador de esta última expresión tienen las siguientes interpretaciones:

- El término de diferencia $\hat{\mathbf{w}}^H(i)\mathbf{u}(i) - \mathbf{w}_o^H\mathbf{u}(i)$ es el error al usar $\hat{\mathbf{w}}^H(i)\mathbf{u}(i)$ como estimación de la magnitud real $\mathbf{w}_o^H\mathbf{u}(i)$. El numerador puede considerarse como la suma de los errores cuadráticos incurridos a lo largo del intervalo $0 \leq i \leq n$.
- El denominador consta de la suma de dos términos: una versión escalada de la distancia euclidiana al cuadrado entre la estimación inicial $\hat{\mathbf{w}}(0)$ y el valor real $\mathbf{w}_o$, y la suma de los cuadrados del ruido $\nu(i)$ a lo largo del mismo intervalo $0 \leq i \leq n$.

La desigualdad anterior implica que la suma de errores al cuadrado está siempre acotada por el efecto combinado de la incertidumbre inicial $\hat{\mathbf{w}}(0) - \mathbf{w}_o$ y el ruido $\nu(i)$.

En conclusión, una realización aislada del algoritmo LMS, más allá de su nombre, no es óptima en sentido medio cuadrático sino en sentido H^∞ .

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado

- En el algoritmo LMS estándar, la corrección que se aplica a $\hat{w}(n+1)$ es proporcional a $u(n)$.
- Para $u(n)$ grande, el algoritmo LMS experimenta un problema de *amplificación del ruido de gradiente*.
- Para contornar este problema es posible utilizar el *algoritmo LMS normalizado*. En particular, la corrección aplicada a $\hat{w}(n+1)$ se normaliza con respecto a la norma Euclidiana al cuadrado de $u(n)$ en la iteración n .
- Es posible formular el algoritmo LMS normalizado como una modificación natural del algoritmo LMS estándar. Alternativamente, es posible obtener el algoritmo LMS normalizado en forma directa, lo que se muestra a continuación.

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- El algoritmo LMS normalizado puede verse como la solución a un problema de optimización con restricciones. Específicamente, el problema de interés puede definirse como sigue:

Dados $u(n)$ y $d(n)$, determinar $\hat{\mathbf{w}}(n+1)$ que minimice la norma Euclidiana al cuadrado del cambio

$$\delta \hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n+1) - \hat{\mathbf{w}}(n)$$

de $\hat{\mathbf{w}}(n+1)$ con respecto a $\hat{\mathbf{w}}(n)$, sujeta a la restricción $\hat{\mathbf{w}}^H(n+1)\mathbf{u}(n) = d(n)$

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- Para resolver este problema de optimización con restricciones utilizaremos el método de *multiplicadores de Lagrange*. La norma al cuadrado del cambio puede expresarse como

$$\begin{aligned}\|\delta\hat{\mathbf{w}}(n+1)\|^2 &= \delta\hat{\mathbf{w}}^H(n+1)\delta\hat{\mathbf{w}}(n+1) \\ &= [\hat{\mathbf{w}}(n+1) - \hat{\mathbf{w}}(n)]^H [\hat{\mathbf{w}}(n+1) - \hat{\mathbf{w}}(n)] \\ &= \sum_{k=0}^{M-1} |\hat{w}_k(n+1) - \hat{w}_k(n)|^2\end{aligned}$$

- Los coeficientes $\hat{w}_k(n) = a_k(n) + jb_k(n)$, $k = 0, 1, \dots, M-1$, se definen en términos de sus partes real e imaginaria.
- Entonces se tiene que

$$\|\delta\hat{\mathbf{w}}(n+1)\|^2 = \sum_{k=0}^{M-1} ([a_k(n+1) - a_k(n)]^2 + [b_k(n+1) - b_k(n)]^2)$$

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- Expresando $u(n) = u_1(n) + ju_2(n)$ y $d(n) = d_1(n) + jd_2(n)$, es posible escribir la restricción como

$$\begin{aligned}\sum_{k=0}^{M-1} (a_k(n+1)u_1(n-k) + b_k(n+1)u_2(n-k)) &= d_1(n) \\ \sum_{k=0}^{M-1} (a_k(n+1)u_2(n-k) - b_k(n+1)u_1(n-k)) &= d_2(n)\end{aligned}$$

- La función costo $J(n)$ puede formularse como

$$\begin{aligned}J(n) &= \sum_{k=0}^{M-1} ([a_k(n+1) - a_k(n)]^2 + [b_k(n+1) - b_k(n)]^2) \\ &\quad + \lambda_1 \left[d_1(n) - \sum_{k=0}^{M-1} (a_k(n+1)u_1(n-k) + b_k(n+1)u_2(n-k)) \right] \\ &\quad + \lambda_2 \left[d_2(n) - \sum_{k=0}^{M-1} (a_k(n+1)u_2(n-k) - b_k(n+1)u_1(n-k)) \right]\end{aligned}$$

donde λ_1 y λ_2 son los *multiplicadores de Lagrange*.

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- Para hallar los valores óptimos de $a_k(n+1)$ y $b_k(n+1)$ diferenciamos $J(n)$ con respecto a esos parámetros e igualamos el resultado a cero. Entonces

$$\frac{\partial J(n)}{\partial a_k(n+1)} = 0 \qquad \frac{\partial J(n)}{\partial b_k(n+1)} = 0$$

lo que produce el siguiente resultado

$$2[a_k(n+1) - a_k(n)] - \lambda_1 u_1(n-k) - \lambda_2 u_2(n-k) = 0$$

$$2[b_k(n+1) - b_k(n)] - \lambda_1 u_2(n-k) - \lambda_2 u_1(n-k) = 0$$

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- Luego, usando la definición de los coeficientes complejos $\hat{w}_k(n)$ es posible combinar estos últimos resultados reales como sigue

$$2[\hat{w}_k(n+1) - \hat{w}_k(n)] = \lambda^* u(n-k), \quad k = 0, 1, \dots, M-1$$

donde $\lambda = \lambda_1 + j\lambda_2$ es un multiplicador de Lagrange complejo.

- Para resolver por la incógnita λ^* , multiplicamos ambos lados de la anterior por $u^*(n-k)$ y luego sumamos entre 0 y $M-1$, para obtener

$$\begin{aligned} \lambda^* &= \frac{2}{\sum_{k=0}^{M-1} |u(n-k)|^2} \left[\sum_{k=0}^{M-1} \hat{w}_k(n+1) u^*(n-k) - \sum_{k=0}^{M-1} \hat{w}_k(n) u^*(n-k) \right] \\ &= \frac{2}{\|\mathbf{u}(n)\|^2} [\hat{\mathbf{w}}^T(n+1) \mathbf{u}^*(n) - \hat{\mathbf{w}}^T(n) \mathbf{u}^*(n)] \end{aligned}$$

donde $\|\mathbf{u}(n)\|$ es la norma Euclidiana de $\mathbf{u}(n)$.

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

Utilizando ahora la restricción compleja en la ecuación anterior es posible obtener

$$\begin{aligned}\lambda^* &= \frac{2}{\|\mathbf{u}(n)\|^2} [d^*(n) - \hat{\mathbf{w}}^T(n) \mathbf{u}^*(n)] \\ &= \frac{2}{\|\mathbf{u}(n)\|^2} e^*(n)\end{aligned}$$

donde se utilizó la definición del error de estimación $e(n)$.

- Finalmente, sustituyendo esta ecuación en la restricción compleja se obtiene

$$\begin{aligned}\delta \hat{w}_k(n+1) &= \hat{w}_k(n+1) - \hat{w}_k(n) \\ &= \frac{1}{\|\mathbf{u}(n)\|^2} u(n-k) e^*(n), \quad k = 0, 1, \dots, M-1 \\ &= \frac{1}{\|\mathbf{u}(n)\|^2} \mathbf{u}(n) e^*(n)\end{aligned}$$

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- Para controlar el cambio del vector de coeficientes de una iteración a la siguiente sin cambiar su dirección, introducimos un factor de escalamiento positivo real denotado por $\bar{\mu}$, tal que

$$\begin{aligned}\delta \hat{w}_k(n+1) &= \hat{w}_k(n+1) - \hat{w}_k(n) \\ &= \frac{\bar{\mu}}{\|\mathbf{u}(n)\|^2} \mathbf{u}(n) e^*(n)\end{aligned}$$

o equivalentemente

$$\hat{w}_k(n+1) = \hat{w}_k(n) + \frac{\bar{\mu}}{\|\mathbf{u}(n)\|^2} \mathbf{u}(n) e^*(n)$$

que determina la ecuación de adaptación para el algoritmo LMS normalizado.

- El aspecto importante a notar es que este algoritmo ajusta $\hat{w}(n+1)$ en términos del *mínimo cambio* (en el sentido de la norma Euclidiana) con respecto a $\hat{w}(n)$.

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

Comparando la recursión del LMS normalizado con la del estándar puede realizarse las siguientes observaciones

- La constante de adaptación $\bar{\mu}$ para el algoritmo LMS normalizado es *adimensional*, mientras que para el estándar tiene dimensiones de *potencia inversa*.

- Haciendo

$$\mu(n) = \frac{\bar{\mu}}{\|\mathbf{u}(n)\|^2}$$

es posible interpretar al algoritmo normalizado como el algoritmo LMS estándar con un *factor de convergencia variante en el tiempo*.

- El algoritmo LMS normalizado es *convergente en media cuadrática* si la constante de adaptación $\bar{\mu}$ satisface

$$0 < \bar{\mu} < 2$$

Algoritmo Least Mean Square (LMS) ...

LMS Normalizado ...

- Otro aspecto importante es que el algoritmo LMS normalizado exhibe una velocidad de convergencia que es potencialmente mayor que la del algoritmo LMS estándar tanto para entradas decorrelacionadas como correlacionadas.
- Aparte del interés en contornar el problema de amplificación de ruido de gradiente, el algoritmo normalizado introduce un problema particular. Específicamente, cuando $\|\mathbf{u}(n)\|$ es pequeña, pueden aparecer dificultades numéricas.
- Para evitar este problema es posible modificar levemente el LMS normalizado como sigue

$$\hat{w}_k(n+1) = \hat{w}_k(n) + \frac{\bar{\mu}}{a + \|\mathbf{u}(n)\|^2} \mathbf{u}(n) e^*(n)$$

donde $a > 0$ es una constante suficientemente pequeña.

Algoritmo Least Mean Square (LMS) ...

LMS normalizado ...

Parámetros: M = número de coeficientes

$\bar{\mu}$ = constante de adaptación tal que

$$0 < \bar{\mu} < 2$$

a = constante positiva

Datos: $\mathbf{u}(n)$ vector de entradas ($M \times 1$).

$d(n)$ respuesta deseada en el instante n .

Inicialización

$$\hat{\mathbf{w}}(0) = \mathbf{0}$$

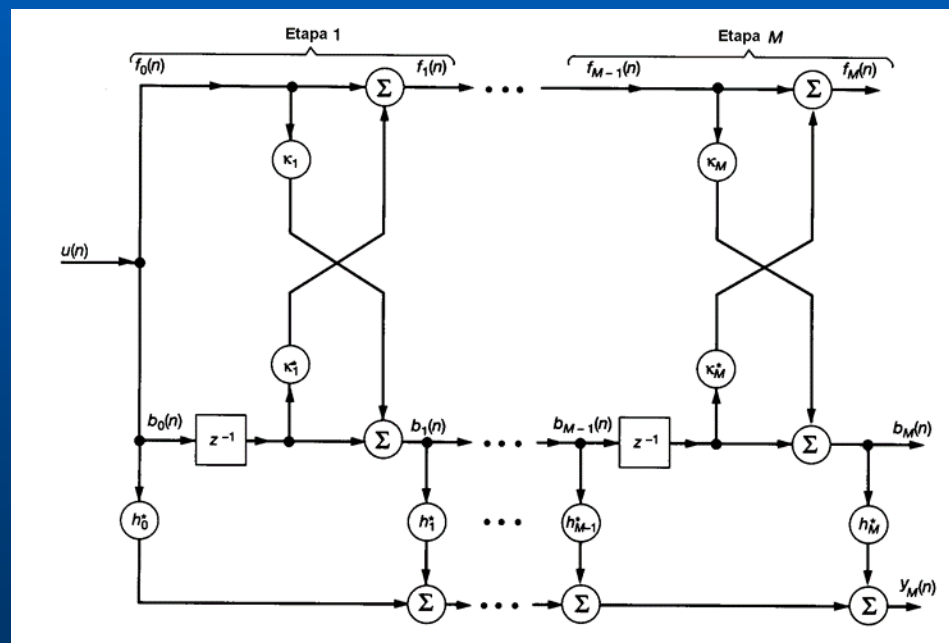
Para cada n , $n = 1, 2, \dots$, calcular

$$e(n) = d(n) - \hat{\mathbf{w}}^H(n)\mathbf{u}(n)$$

$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \frac{\bar{\mu}}{a + \|\mathbf{u}(n)\|^2} \mathbf{u}(n)e^*(n)$$

Algoritmo de gradiente lattice (Gradient-adaptive lattice, GAL)

Otro algoritmo de tipo **gradiente estocástico** utiliza la idea del **estimador conjunto**, en base a un predictor lattice multietapas y un combinador lineal.



Algoritmo de gradiente lattice (GAL) ...

En relación al predictor lattice (se asume que la entrada es estacionaria en sentido amplio), las etapas están caracterizadas por el **coeficiente de reflexión** κ_m (complejo). Para su estimación, se considera la función costo

$$J_{fb,m} = \frac{1}{2} E\{|f_m(n)|^2 + |b_m(n)|^2\}$$

donde, del estimador conjunto de predicción lineal, se tiene que

$$f_m(n) = f_{m-1}(n) + \kappa_m^* b_{m-1}(n-1) \quad b_m(n) = b_{m-1}(n-1) + \kappa_m f_{m-1}(n)$$

Y sustituyendo en la anterior y derivando se obtiene

$$\frac{\partial J_{fb,m}}{\partial \kappa_m} = \kappa_m (E\{|f_{m-1}(n)|^2 + |b_{m-1}(n)|^2\}) + 2E\{b_{m-1}(n-1)f_{m-1}^*(n)\}$$

tal que, igualando a 0

$$\kappa_{m,o} = -\frac{2E\{b_{m-1}(n-1)f_{m-1}^*(n)\}}{E\{|f_{m-1}(n)|^2 + |b_{m-1}(n)|^2\}}$$

Algoritmo de gradiente lattice (GAL) ...

Finalmente, asumiendo como un estimador adecuado usando promedios temporales (formula de Burg)

$$\hat{\kappa}_m(n) = -\frac{2 \sum_{i=1}^n [b_{m-1}(i-1) f_{m-1}^*(i)]}{\sum_{i=1}^n [|f_{m-1}(i)|^2 + |b_{m-1}(i)|^2]}$$

Esto define un estimador en bloques de κ_m . Para obtener un estimador recursivo, se parte de la un estimador de la energía en la etapa m del predictor con

$$\begin{aligned} \mathcal{E}_{m-1}(n) &= \sum_{i=1}^n (|f_{m-1}(i)|^2 + |b_{m-1}(i-1)|^2) \\ &= \mathcal{E}_{m-1}(n-1) + (|f_{m-1}(n)|^2 + |b_{m-1}(n-1)|^2) \end{aligned}$$

Útil para obtener una forma recursiva del denominador del estimador. Para el numerador es posible tener en cuenta también

$$\sum_{i=1}^n b_{m-1}(i-1) f_{m-1}^*(i) = \left[\sum_{i=1}^{n-1} b_{m-1}(i-1) f_{m-1}^*(i) \right] + b_{m-1}(n-1) f_{m-1}^*(n)$$

Algoritmo de gradiente lattice (GAL) ...

Con lo que , reordenando y sustituyendo en el estimador se obtiene

$$\hat{\kappa}_m(n) = \hat{\kappa}_m(n-1) - \frac{f_{m-1}^*(n)b_{m-1}(n) + b_{m-1}(n-1)f_m^*(n)}{\mathcal{E}_{m-1}(n)}$$

Para contemplar un contexto no estacionario, la anterior se modifica la siguiente forma

$$\hat{\kappa}_m(n) = \hat{\kappa}_m(n-1) - \frac{\tilde{\mu}}{\mathcal{E}_{m-1}(n)}(f_{m-1}^*(n)b_{m-1}(n) + b_{m-1}(n-1)f_m^*(n))$$

donde ahora

$$\mathcal{E}_{m-1}(n) = \beta \mathcal{E}_{m-1}(n-1) + (1-\beta)(|f_{m-1}(n)|^2 + |b_{m-1}(n-1)|^2)$$

y $\tilde{\mu}$ es el factor de convergencia y $0 \leq \beta \leq 1$.

Algoritmo de gradiente lattice (GAL) ...

El estimador conjunto se completa con la estimación de la respuesta deseada $d(n)$ a través de un combinador de los errores de predicción $b_0(n), b_1(n), \dots, b_m(n)$ con los parámetros $\hat{h}_0(n), \hat{h}_1(n), \dots, \hat{h}_m(n)$,

$$y_m(n) = \sum_{k=0}^m \hat{h}_k^*(n) b_k(n) = y_{m-1}(n) + \hat{h}_m^*(n) b_m(n)$$

Tal que, definiendo el error de estimación como $e_m(n) = d(n) - y_m(n)$, se tiene el siguiente algoritmo

$$\hat{h}_m(n+1) = \hat{h}_m(n) - \frac{\tilde{\mu}}{\|\mathbf{b}_m(n)\|^2} b_m(n) e_m^*(n)$$

Algoritmo Gradient-adaptive lattice (GAL)

Parámetros:

M = orden del predictor; β = constante, $(0 < \beta < 1)$

$\tilde{\mu} < 0.1$ = constante de adaptación; δ, a = constantes positivas

Predictor lattice multietapa

Para el predictor de orden $m = 1, 2, \dots, M$, hacer

$$f_m(0) = b_m(0) = 0; \mathcal{E}_{m-1}(0) = a; \hat{\kappa}_m(0) = 0$$

Para las muestras $1, 2, \dots$, hacer

$$f_0(n) = b_0(n) = u(n), u(n) = \text{entrada del predictor lattice}$$

Para el predictor de orden $m = 1, 2, \dots, M$, y las muestras $1, 2, \dots$, calcular

$$\mathcal{E}_{m-1}(n) = \beta \mathcal{E}_{m-1}(n-1) + (1-\beta)(|f_{m-1}(n)|^2 + |b_{m-1}(n-1)|^2)$$

$$f_m(n) = f_{m-1}(n) + \hat{\kappa}_m^*(n-1)b_{m-1}(n-1)$$

$$b_m(n) = b_{m-1}(n-1) + \hat{\kappa}_m(n-1)f_{m-1}(n)$$

$$\hat{\kappa}_m(n) = \hat{\kappa}_m(n-1) - \frac{\tilde{\mu}}{\mathcal{E}_{m-1}(n)}[f_{m-1}^*(n)b_m(n) + b_{m-1}(n-1)f_m^*(n)]$$

Estimador de la respuesta deseada

Para el predictor de orden $m = 1, 2, \dots, M$, hacer

$$\hat{h}_m(0) = 0$$

Para las muestras $1, 2, \dots$, hacer

$$y_{-1}(n) = 0, \|\mathbf{b}_{-1}(n)\|^2 = \delta$$

Para el predictor de orden $m = 1, 2, \dots, M$, y las muestras $1, 2, \dots$, calcular

$$y_m(n) = y_{m-1}(n) + \hat{h}_m^*(n)b_m(n)$$

$$e_m(n) = d(n) - y_m(n)$$

$$\|\mathbf{b}_m(n)\|^2 = \|\mathbf{b}_{m-1}(n)\|^2 + |b_m(n)|^2$$

$$\hat{h}_m(n+1) = \hat{h}_m(n) + \frac{\tilde{\mu}}{\|\mathbf{b}_m(n)\|^2} b_m(n) e_m^*(n)$$

Algoritmo Gradient-adaptive lattice (GAL) ...

Algunas propiedades

- El uso del parámetro de tamaño de paso variable en el tiempo $\tilde{\mu}$ introduce una **forma de normalización** similar a la del algoritmo LMS normalizado.
- Anteriormente, nos referimos al algoritmo GAL como de naturaleza aproximada ya que no hay garantía que, en un entorno no estacionario, la **propiedad de desacoplamiento** (es decir, la ortogonalidad de los errores de predicción inversa) se conserve.
- El resultado neto es que, aunque el comportamiento de convergencia del algoritmo GAL es **más rápido que el del algoritmo LMS**, es inferior a los algoritmos de red de mínimos cuadrados recursivos, aunque de menor complejidad.

Menor Cuadrado Mínimo (LMS)

Método de Steepest Descent

- Algoritmo Steepest Descent
- Estabilidad del algoritmo

Algoritmo LMS

- Ajuste de los coeficientes
- Estabilidad y desempeño del algoritmo LMS
- El algoritmo LMS normalizado

Filtrado adaptativo en el dominio frecuencia

- Filtrado adaptativo en bloques
- Algoritmo adaptativo con preprocesamiento
- Clasificación de algoritmos